NAME – DUSHYANT SINGH

CSE-4E

REG. NO - 202401100200280138

DEVOPS - MSE2

Test Case 1 — Linux System Audit & User Management

Scenario: You are a Linux system administrator onboarding new employees at a tech company. Complete the following tasks on your system.

Tasks

1. Check the identity of the currently logged-in user and display the username of the current session.
2. Check which users are currently logged into the system and what they are doing.
3. View the login history of the system.
4. Display complete system information, the hostname, and how long the system has been running.
5. Create two groups: developers and qa. Verify both groups were successfully created and display the complete list of groups on the system.
6. Create the following users and assign each to their respective group:

Employee	Department
aman	developers
ritu	developers
karan	qa

7. Employee ritu has resigned. Ensure she has no active sessions, delete her account along with her home directory, and verify she no longer exists on the system.

Expected Outcome

- System audit commands display current user, active sessions, login history, hostname, and uptime without errors.
- Groups developers and qa appear in the system group list.
- Users aman and karan are created and assigned to the correct groups (verified via id command).
- After deletion, any attempt to look up ritu returns: **"no such user"**.

1. Check current logged-in user

whoami

2. Check currently logged-in users and their activity

who

Detailed info

w

3. View login history

last

4. Display system information, hostname, and uptime

System information

```
uname -a

# Hostname
hostname

# Uptime
uptime
```

5. Create groups: developers and qa

```
sudo groupadd developers
sudo groupadd qa

# Verify groups created
getent group developers
getent group qa
```

Display all groups

```
cat /etc/group
```

6. Create users and assign to groups

```
# Create aman in developers
sudo useradd -m -g developers aman
```

Create ritu in developers

sudo useradd -m -g developers ritu

Create karan in qa

sudo useradd -m -g qa karan

Verify user group assignment

id aman

id ritu

id karan

7. Remove user ritu after resignation

Delete user along with home directory

sudo userdel -r ritu

Verify deletion

id ritu

Test Case 2 — File Permission Configuration

Scenario: A web server requires a properly configured index.html file with specific ownership and access permissions.

Tasks

1. Navigate to your home directory and create a file named index.html.
2. Add any HTML text content to the file.
3. Check the file's current default permissions.
4. Grant full permissions (read, write, execute) to all users on the file.
5. Change the file owner and group to www-data.
6. Change the file permissions to 755.
7. Display the final permissions and ownership to verify the configuration.

Expected Outcome

- File index.html exists in the home directory with content.
- After step 4, permissions show rwxrwxrwx.
- After step 5, owner and group both show www-data.
- Final ls -l output shows:

-rwxr-xr-x 1 www-data www-data ... index.html

```
# 1. Navigate to home directory
cd ~

# 2. Create index.html file
touch index.html

# 3. Add HTML content to the file
echo "<html><body><h1>Hello World</h1></body></html>" > index.html

# 4. Check current default permissions
ls -l index.html
```

```
# 5. Grant full permissions to all users
chmod 777 index.html

# Verify permissions
ls -l index.html


# 6. Change owner and group to www-data
sudo chown www-data:www-data index.html

# Verify ownership
ls -l index.html


# 7. Change permissions to 755
chmod 755 index.html

# 8. Display final permissions and ownership
ls -l index.html
```

Test Case 3 — Node.js Application Setup & Git Integration

Scenario: You are setting up a new Node.js REST API from scratch and publishing it to GitHub.

Tasks

1. Initialize a new Node.js project and install the required dependencies: express and @types/express (as a dev dependency).
2. Create an index.js file that starts an Express server on port 8080 with the following endpoint:
    - GET /health — returns HTTP status 200 with the JSON response { "status": "OK" }.
3. Initialize a Git repository in the project folder.
4. Create an initial commit with all project files.
5. Create a GitHub repository named nodejs-api and push your code to it.

Expected Outcome

- Project folder contains package.json, node_modules/, and index.js.
- Running the server locally and calling GET /health returns:

{ "status": "OK" }

- GitHub repository nodejs-api is publicly visible with at least one commit containing all project files.

```
# ========================================
# NODE.JS APPLICATION SETUP & GIT INTEGRATION
# ========================================

# 1. Create project folder and navigate into it
mkdir nodejs-api
cd nodejs-api

# 2. Initialize Node.js project
npm init -y
```

```
# 3. Install dependencies
npm install express
npm install --save-dev @types/express

# 4. Create index.js file
touch index.js

# 5. Add Express server code
# Open index.js and paste the following:

const express = require('express');
const app = express();
const PORT = 8080;

app.get('/health', (req, res) => {
   res.status(200).json({ status: "OK" });
});

app.listen(PORT, () => {
   console.log(`Server running on http://localhost:${PORT}`);
});


# 6. Run the server
node index.js
```

```
# Test endpoint:
# Open browser or use curl:
# curl http://localhost:8080/health

# Expected Output:
# { "status": "OK" }


# ================================
# GIT SETUP
# ================================

# 7. Initialize Git repository
git init

# 8. Add all files
git add .

# 9. Commit files
git commit -m "Initial commit - Node.js API setup"


# ================================
# GITHUB INTEGRATION
# ================================
```

```
# 10. Create a repository on GitHub named: nodejs-api

# 11. Add remote origin (replace with your repo URL)
git remote add origin https://github.com/your-username/nodejs-api.git

# 12. Push code to GitHub
git branch -M main
git push -u origin main


# ================================
# EXPECTED OUTPUT
# ================================

# Files in project:
# package.json
# node_modules/
# index.js

# API Response:
# { "status": "OK" }

# GitHub repo:
# nodejs-api (public) with at least one commit
```

==========

END OF TASK

==========